Coding Saint

The Nirvana Guide to Spring Boot

Coding Saint

Introduction **3**
Requirements and Prerequisites 3
Developing first spring boot application 4
@SpringBootApplication ,@RestController , @EnableAutoConfiguration and @RequestMapping 6
@SpringBootApplication 7
@SpringBootConfiguration 7
@EnableAutoConfiguration 7
@ComponentScan 8
@RestController 8
@RequestMapping 8

2. Working RESTful API **13**
Database Configuration 13
A User RESTful API 13
If you see above interface we annotated it with @Repository to make it a Spring bean and also to demote that this bean does an database related operation. Further we extented it to JPARepository enabling it to do CRUD operations. Now JPARepository has methods like save, get, getAll, delete which will be available to UserRepository by default. 15
Rest Controller 15

3. Configuration and Properties **16**
Spring Boot as said is having an opinionated approach. It comes up with range of pre configuration options. Once we initialize application we have application.properties file available in our application. Since application.properties is already at classpath , Spring Boot allows you to use it for any configuration, be it yours or overriding existing spring boot configuration. 16
@Value 17
Autowiring Environment to get application.properties 18
Externalizing Configuration 18
Type Safe Configuration Properties 27
@ConfigurationProperties 27

4. Profiles **29**
Using Properties file for profiles. 29
Adding Active Profiles 33

5. Logging **36**

File Output 37
Log Level 37
Profile specific configurations 37
Environment properties 39

6. Developing a web application 41
Introduction to Web MVC Framework 41
Spring MVC Autoconfiguration 41
Disabling auto configuration 41
HttpMessageConverter 42

1. Introduction

Spring Boot is easy, robust and simple way to create production grade standalone application that you can run.

It is a very opinionated approach of Spring boot library and third party application with a motive of starting an application without much configuration. Spring Boot handles most of the configuration required for Spring and third party libraries.

Tough it handles most of your configuration by itself , it's opinionated approach doesn't come in your way to override default configuration , and you can still add custom configuration on your own.

Running a Spring Boot application is as easy as running a jar file.

a. Requirements and Prerequisites

While writing this documentation spring boot is at version 2.0.0.BUILD-SNAPSHOT. It requires

1. Java Version 8
2. Maven 3.2 + or Gradle 4
3. Servlet Container as Tomcat 8.5 /Jetty 9.4 /Undertow 1.3

While proceeding further we do expect that you have Java version 8, Maven 3.2 or Gradle installed on your system.

It is expected that as a reader you do have a basic knowledge of Java and Maven . A knowledge on Spring is a big plus but you can learn it along the course. Things will be kept as simple as possible, with links to full source code.

b. Developing first spring boot application

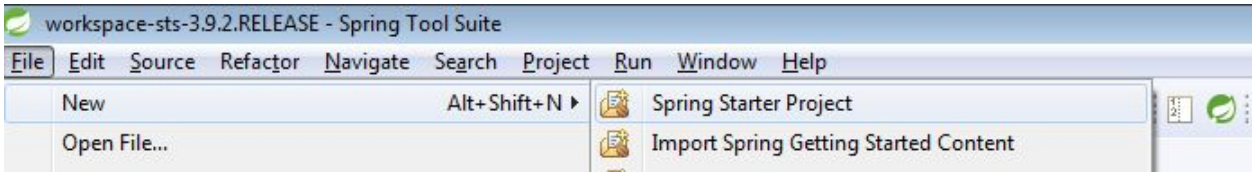
Spring Boot currently support Java, Kotlin and Groovy as language preference to create an application. You can also choose Maven or Gradle as a build automation tool. You can choose your favourite IDE ranging from Netbeans, IntelliJ ,Eclipse or Spring Tool Suite.

We will be using Java and Maven with Spring Tool Suite.

We can create a Spring Boot application in different ways.

- ❑ Creating a maven project and adding dependencies
- ❑ Using Spring Tool Suite (STS)/IntelliJ built in feature
- ❑ Using Spring Initializr : https://start.spring.io

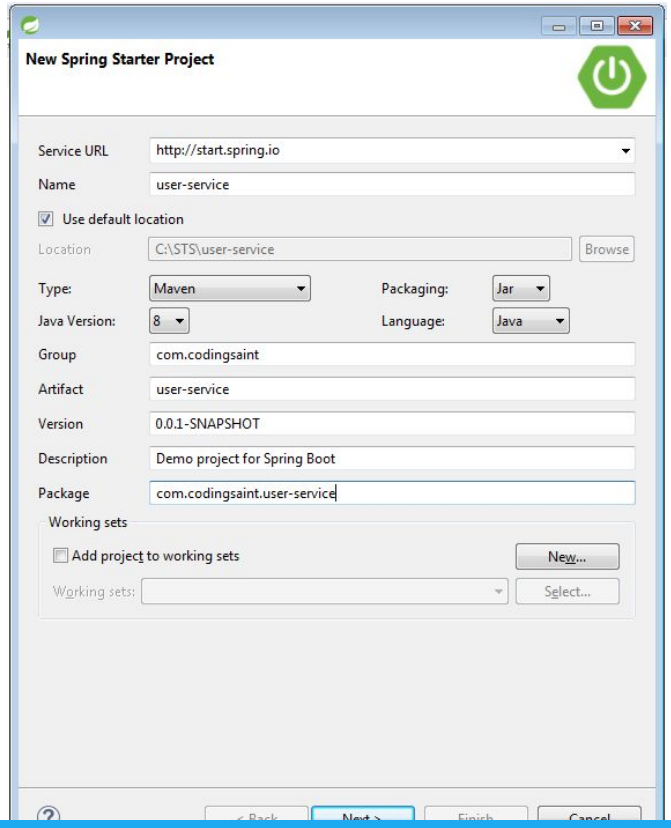
We will be using STS built in feature and will also look at Spring Initializr. In order to create first spring boot application using STS , click on File ➔ New ➔ Spring Starter Project.



It will open a popup , where default values can be modified.

We have modified name, group ,artifact and package default values.

Once modified as per your requirement ,click “Next”.

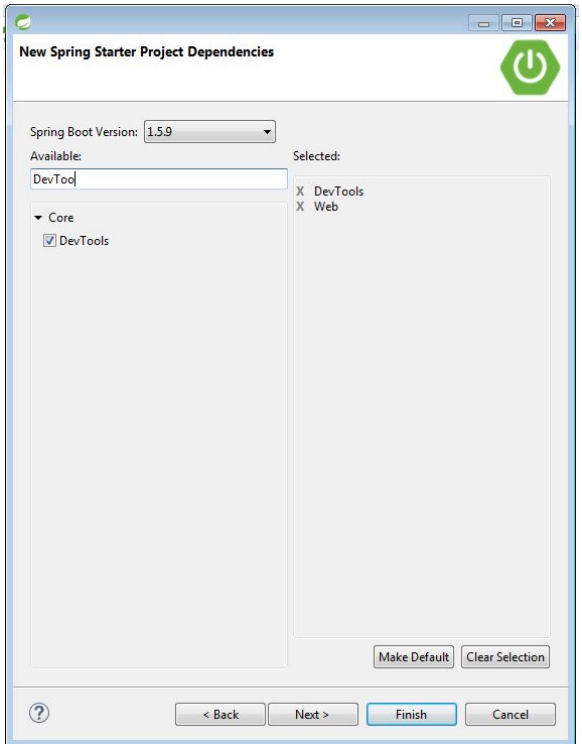


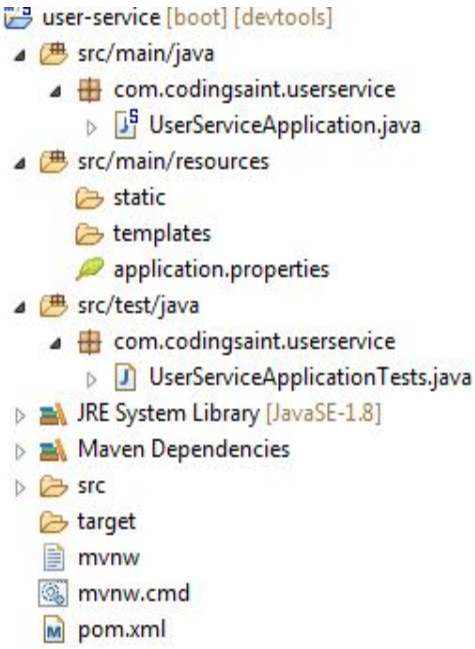
Now select Web from web stack and Devtools from Core stack.

Selecting Dev Tool is optional. It is just because of personal choice of not starting application again and again on saving the files. We will learn about DevTool later.

Web is an obvious choice as we are going to learn a web based Spring Boot application.

Click Finish. Now we have a sample Spring boot application ready to go.





After clicking on finish , we will have project structure as shown in figure below.

Here UserServiceApplication.java inside com.codingsaint.userservice is the main class and entry point to our application.

Every Spring Boot Application has a main class, which acts as the entry point to application.

Let's run it and see if Spring Boot application is really this easy to start and run. You can run it as a Java Application by right clicking on file and selecting Run as from menu and selecting " Java Application" or "Spring Boot App" as an option.

Once you run it , you can see output as below on console ,where it does tell that application is running on port 8080.

```
8-01-15 15:54:25.419  INFO 8780 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'hiddenHttpMethodFilter' to: [/*]
8-01-15 15:54:25.419  INFO 8780 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'httpPutFormContentFilter' to: [/*]
8-01-15 15:54:25.419  INFO 8780 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'requestContextFilter' to: [/*]
8-01-15 15:54:26.071  INFO 8780 --- [  restartedMain] s.w.s.m.m.a.RequestMappingHandlerAdapter : Looking for @ControllerAdvice: org.springframework.boot.context.embedded.An
8-01-15 15:54:26.165  INFO 8780 --- [  restartedMain] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/error]}" onto public org.springframework.http.ResponseEntity<jav
8-01-15 15:54:26.165  INFO 8780 --- [  restartedMain] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/error],produces=[text/html]}" onto public org.springframework.wel
8-01-15 15:54:26.228  INFO 8780 --- [  restartedMain] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/webjars/**] onto handler of type [class org.springframewo
8-01-15 15:54:26.228  INFO 8780 --- [  restartedMain] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/**] onto handler of type [class org.springframework.web.s
8-01-15 15:54:26.306  INFO 8780 --- [  restartedMain] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/**/favicon.ico] onto handler of type [class org.springfra
8-01-15 15:54:26.493  INFO 8780 --- [  restartedMain] o.s.b.d.a.OptionalLiveReloadServer       : LiveReload server is running on port 35729
8-01-15 15:54:26.571  INFO 8780 --- [  restartedMain] o.s.j.e.a.AnnotationMBeanExporter        : Registering beans for JMX exposure on startup
8-01-15 15:54:26.696  INFO 8780 --- [  restartedMain] s.b.c.e.t.TomcatEmbeddedServletContainer : Tomcat started on port(s): 8080 (http)
8-01-15 15:54:26.712  INFO 8780 --- [  restartedMain] c.c.userservice.UserServiceApplication   : Started UserServiceApplication in 4.716 seconds (JVM running for 9.536)
```

In browser open http://localhost:8080/ and verify if server is up. Following output with 404 (default 404 of Spring Boot application) tells that server is up and running and obviously we have nothing mapped to be displayed for base URL so we are seeing this 404 error page.

In Next section , we will see how to handle incoming requests properly. We will also dive in Spring Boot Basics.

@SpringBootApplication ,@RestController , @EnableAutoConfiguration and @RequestMapping

All Spring Boot Application have one class with main method as an entry point. Our application also has one , It's UserServiceApplication.java under com.codingsaint.userservice package. If we look at code of this class.

```
package com.codingsaint.userservice;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class UserServiceApplication {
        public static void main(String[] args) {
                SpringApplication.run(UserServiceApplication.class, args);
        }
}
```

Here , we see the class with main as an entry point and annotated with @SpringBootApplication.

@SpringBootApplication

A spring boot application starts from the class annotated with @SpringBootApplication ,so there must be one and at most one class with main method signature annotated with @SpringBootApplication.

@SpringBootApplication is the entry point of any Spring Boot application, This annotation is actually a combination of few other annotation, @SpringBootConfiguration , @EnableAutoConfiguration and @ComponentScan

@SpringBootConfiguration

It tells that this class acts as a configuration class and can be used alternatively for @Configuration which is another standard annotation of Spring

@EnableAutoConfiguration

It indicates that Spring will guess automatically the configuration classes that application will need and configure them. This is the reason Spring boot is said to have a very opinionated

approach. Tough it's opinionated approach doesn't stop you to configure the way you wanted.EnableAutoConfiguration looks at the classpath to find the beans you would require at runtime and configure them.

@ComponentScan

This annotation helps you to scan all Spring beans. It searches bean in packages you ask it to search, provided via value to annotation viz @ComponentScan({"my.awesome.package"}).

@SpringBootApplication with inbuilt @ComponentScan searches the packages in which main class is, and the packages below it. It means with our application structure it will search any class which is in com.codingsaint.userservice and sub packages i.e. com.codingsaint.userservice.* like com.codingsaint.userservice.vo or com.codingsaint.userservice.dao etc.

Now let's make our application serve some web url requests . Since will start with a RESTful application , we will create controllers and map requests to its methods. We will ensure that if someone sends a URL request and it is valid , our application sends a valid response.

@RestController

To start let's make a Controller , Spring gives a specific controller i.e. @RestController to ensure that all methods within it , returns a REST enabled response. It automatically marshalls and unmarshal incoming JSON/XML request to Java object and sends response of Java objects as JSON/XML output.

We will create a class UserController under package com.codingsaint.userservice.controller and annotate it with @RestController.

```
package com.codingsaint.userservice.controller;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class UserController {
}
```

Here RestController is denotes that this class has to act as a Controller and server incoming requests. The method within it , will be mapped with @RequestMapping to map the requests.

@RequestMapping

@RequestMapping is used for mapping incoming requests to specific method. Obviously for each request , we should have a dedicated unique method. In web application , a request is made of

URL path , HTTP verbs like GET , PUT ,POST ,DELETE etc and headers . Each URL with combination of these verb and headers(if added) is unique and for each unique request we must have a unique method handling it.

For example , let's consider a URL as "*/v1/user/{id}*" . Here v1 is our api version , and id is the unique it of the user, If we use GET HTTP verb along with the URL to hit the server , it should be considered as request for getting user information with the provided id. At the same time if we use DELETE , it should be considered as deleting the details of user with same id. In both cases we should have a dedicated method annotated with @RequestMapping the handling incoming request.

@RequestMapping can be used at class level as well as on method level. Once used on class level , it is appended to all the method level request mappings. For example , in above method created UserController class we are supposed to map requests for user where version of our API will be v1 , so if we map v1 at class level , we don't have to map it at all methods inside the class used for mapping URLs.

Let is create a mapping for getting all users but before that we need a user object. We will create a simple POJO class with id , first name, last name and email. We will create User.java inside com.codingsaint.userservice.models , where models denoting package for our model classes.

```
package com.codingsaint.userservice.models;
public class User {
        private Long id;
        private String firstName;
        private String lastName;
        private String email;
//Getters and setters : Do add them
}
```

In above class we have removed getters and setters for readability , please add them.

Now we will add @RequestMapping at class and methods levels to handle incoming request.

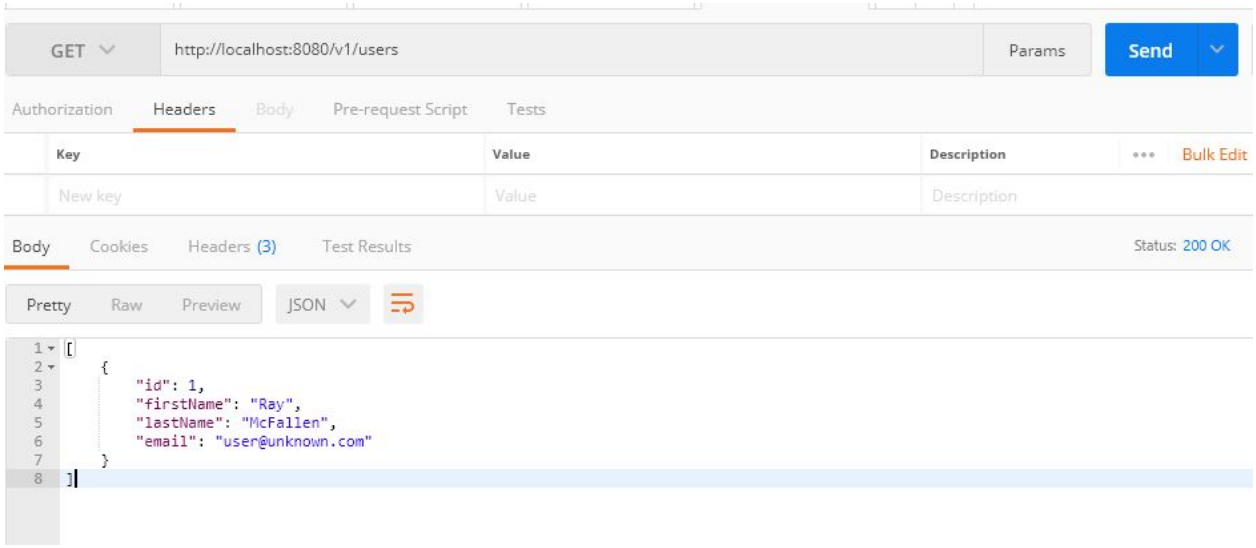
```
@RestController
@RequestMapping("v1")
public class UserController {
        private List<User> users = new ArrayList<User>();
        @RequestMapping(value = "users", method = RequestMethod.GET)
        protected List<User> getUsers() {
                User user = new User();
```

```
            user.setId(1L);
            user.setEmail("user@unknown.com");
            user.setFirstName("Ray");
            user.setLastName("McFallen");
            users.add(user);
            return users;
        }
}
```

Here we don't have a database yet, so we have taken a List and initialize it with a dummy user. Now after running UserServiceApplication.java (as a java program/ as spring boot app),with POSTMAN ,ARC (chrome plugins) or with a CURL request , we can see a valid response. Currently if we run it , the application will be up at http://localhost:8080/ and with URL request mapped at http://localhost:8080/v1/users , we get following result.

```
[
    {
        "id": 1,
        "firstName": "Ray",
        "lastName": "McFallen",
        "email": "user@unknown.com"
    }
]
```

Following is a screenshot from POSTMAN

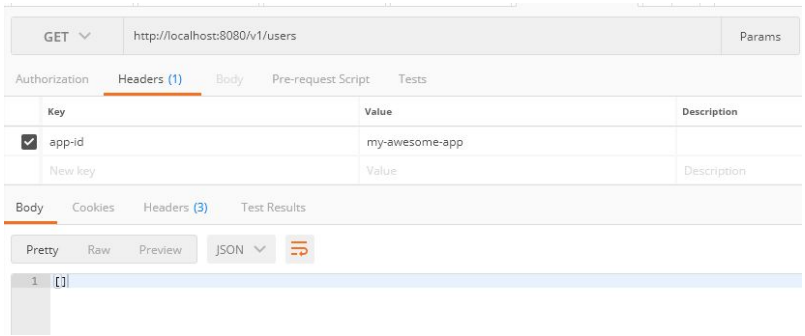


RequestMapping annotation by default works as explained but it can be used with more parameters like consumes, produces, path, name ,headers and params ,path and value are alias

of each other. We can have two different methods handing same signature based on URL ,HTTP Verb but different acceptance header.

Let's add another method as below

```
@RequestMapping(value = "users", method = RequestMethod.GET ,headers =
"app-id=my-awesome-app")
      protected List<User> getUsersWithAppIdHeader() {
            return new ArrayList<User>();
      }
```



With addition of this method we can send one request as above and one with header value , app-id=my-awesome-app and both will be resolved separately. Below is the screenshot of same , sending empty list as response.

It is also interesting to note that value, header, consumes ,accepts all accept array , so we can send multiple values to it , which means we can map multiple URL signature to one method but not exactly same URL signature to multiple method which will confuse application to hit which method in case of incoming request.

Creating a JAR

In Spring Boot application , inside pom file we have following build dependency which enables our application to create a JAR.

```
<build>
            <plugins>
                  <plugin>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-maven-plugin</artifactId>
                  </plugin>
            </plugins>
      </build>
```

The above plugin helps maven to build a runnable JAR. In order to do so , type maven package command from prompt and a jar will be build.

```
mvn package
```

```
[INFO] --- maven-jar-plugin:2.6:jar (default-jar) @ user-service ---
[INFO] Building jar: F:\STS_Workspace\codingsaint\user-service\target\user-service-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot-maven-plugin:1.5.9.RELEASE:repackage (default) @ user-service ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time: 26.104 s
[INFO] Finished at: 2018-01-22T08:46:12+05:30
[INFO] Final Memory: 22M/72M
[INFO] ------------------------------------------------------------------------
```

In above screenshot , we can see that a jar is build at target folder. We can run the Jar and even ship it as standalone application. It has embedded tomcat, so we don't require a seperate tomcat instance to deploy it.

To run it as standalone application run with command :-

java -jar user-service-0.0.1-SNAPSHOT.jar

```
rk.boot.autoconfigure.web.BasicErrorController.error(javax.servlet.http.HttpServletRequest)
2018-01-22 08:52:35.760  INFO 7176 --- [           main] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/we
bjars/**] onto handler of type [class org.springframework.web.servlet.resource.ResourceHttpRequestHandler]
2018-01-22 08:52:35.767  INFO 7176 --- [           main] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/**
] onto handler of type [class org.springframework.web.servlet.resource.ResourceHttpRequestHandler]
2018-01-22 08:52:35.847  INFO 7176 --- [           main] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/**
/favicon.ico] onto handler of type [class org.springframework.web.servlet.resource.ResourceHttpRequestHandler]
2018-01-22 08:52:36.229  INFO 7176 --- [           main] o.s.j.e.a.AnnotationMBeanExporter        : Registering beans fo
r JMX exposure on startup
2018-01-22 08:52:36.853  INFO 7176 --- [           main] s.b.c.e.t.TomcatEmbeddedServletContainer : Tomcat started on po
rt(s): 8080 (http)
2018-01-22 08:52:36.886  INFO 7176 --- [           main] c.c.userservice.UserServiceApplication   : Started UserServiceA
pplication in 13.762 seconds (JVM running for 15.447)
```

2. Working RESTful API

RESTful APIs have been all over the web development these days. RESTful API enables easy communication between systems using JSON/XML over age old HTTP protocols. This makes them easy and familiar to use with minimum learning curve.

With the knowledge of Basic annotation let's make a working API . Here we will use H2 embedded database for our use and will try to perform CRUD using RESTful operations.

Database Configuration

Let us start by configuring dataSource . As we know Spring boot has an opinionated approach , it does provide simple application properties to create a dataSource.

```
# H2
spring.h2.console.enabled=true
spring.h2.console.path=/h2
# Datasource
spring.datasource.url=jdbc:h2:file:~/user
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
```

The first two properties are used for enabling H2 console. Since it is a embedded database these property allow us to use a UI to see the database and run queries. When system will start it will ensure that we can see the database at `http://localhost:port_number/h2`

After that we are using few default properties for creating data source. These are pretty standard properties to crete any dataSource i.e. database URL ,username ,password and driver. Here we are using H2 related configuration.

A User RESTful API

Let's start with creating an API for creating,updating,retrieving and deleting a User for an Application. We will keep things simple and below is our domain model for User

```java
@Entity
public class User implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String firstName;
    private String lastName;
    private String email;

    public User() {
        super();
    }

   //getters and setters
}
```

We are using few annotation from JPA to use it for inserting in database.

@Entity : Tells there should be a table for User object.

@Id : denotes that the field will be used as a primary key.

@GeneratedValue: Let's decide how that primary key will be generated , in our case it is GenerationType.AUTO which means it will be automatically generated with increments.

Now once we have a full User object and Entity , let's see how our easily and magically we will be adding a way to operate database related operation.

The Repositories

We will be creating a simple repository which will help us to talk to database and do User related operation. Using JPA we can create an interface say UserRepository and that will help us to perform CRUD operations over User table.

```java
package com.codingsaint.tutorial.spring.userservice.repository;

import com.codingsaint.tutorial.spring.userservice.model.User;
@Repository
public interface UserRepository extends JpaRepository<User, Long>{

}
```

If you see above interface we annotated it with @Repository to make it a Spring bean and also to demote that this bean does an database related operation. Further we extented it to JPARepository enabling it to do CRUD operations. Now JPARepository has methods like save, get, getAll, delete which will be available to UserRepository by default.

Let's autowire it to a Controller and handle incoming HTTP requests.

Rest Controller

Now we will create a controller which will help us to communicate with clients . Let's start with saving an object.Below is sample Rest Controller we are using.

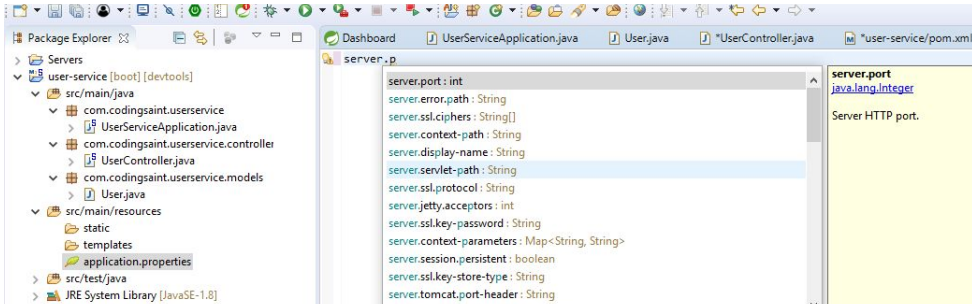
```
@RestController
public class UserController {
	@Autowired
	private UserRepository userRepository;

	@RequestMapping(path = { "/user" }, method = RequestMethod.POST)
	public ResponseEntity<Void> add(@RequestBody User user) {
		ResponseEntity<Void> response = null;
		try {
			userRepository.save(user);
			response = new ResponseEntity<>(HttpStatus.OK);
		} catch (Exception e) {
			response = new
ResponseEntity<>(HttpStatus.EXPECTATION_FAILED);
		}
		return response;
	}
}
```

3. Configuration and Properties

Spring Boot as said is having an opinionated approach. It comes up with range of pre configuration options. Once we initialize application we have application.properties file available in our application. Since application.properties is already at classpath , Spring Boot allows you to use it for any configuration, be it yours or overriding existing spring boot configuration.

Let's see one simple example , In our last chapter we created a sample spring boot application. It has a embedded tomcat server. Spring Boot has an opinion that tomcat server should use 8080 as default port. In reality it will not be always possible to run our application on 8080 , there could be cases where 8080 is already occupied by some other application or we by some reason want it to run on some port other than 8080. We have file application.properties which we can make use of , spring boot provides a property called "server.port" if added to application.properties file will override Spring boot opinion of 8080.

Let's add server.port in application.properties file.




```
server.port=8081
```

Once you add above property in spring boot application properties and server starts, you can see it running at 8081 now. In above figure you can also see , Spring Tool Suite suggesting extra possible value. These are defaults which come with spring boot application and spring has given us the chance to override these defaults. We can find the list of all defaults at

https://docs.spring.io/spring-boot/docs/current/reference/html/common-application-properties.html

We can simply configure from server port to database settings (like dataSource) by adding values in application.properties. We can add even custom property and use it. We will be using many of the available properties as we go along this book.

Let us also have a look on how can we use application.properties further and can get the values from properties file to our java classes.

@Value

As we have seen that we can define our properties at application.properties , there should be a easy way to get this values at our java files. @Value is the annotation to help us.

Let's see how can we use it. Let's define one property spring.application.name , this is a default Spring boot property (already given by Spring Boot) , we will try to access it in our controller using @Value annotation.

We will add the line below at application.properties file

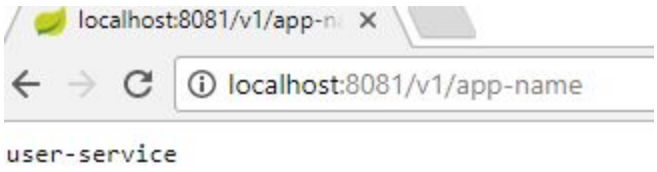
```
spring.application.name=user-service
```

Once added let's add a string named applicationName to our controller , We will also also add a method to access it , say applicationName which will only return us the application name.We will use existing UserController to test, Add following code snippet insite UserController.

```
@Value("${spring.application.name}")
        private String applicationName;
        @RequestMapping("app-name")
        protected String getApplicationName() {
                return applicationName;
        }
```

In above code we are not initializing applicationName variable , yet we will be able to get the value which we have added in application.properties via @Value annotation.

If we hit URL we will get the value for spring.application.name bound to applicationName variable



Autowiring Environment to get application.properties

As we have have received the value via @Value annotation , there could be a case where we have large number or properties to be read, It would be difficult to define so many variables and reading them via @Value annotation, we can autowire Environment(org.springframework.core.env.Environment) class to achieve same result.

Let's autowire it.

```
@Autowired
private Environment env;
```

Now to access our defined variable in application.properties try getProperty method of Environment class as below

```
@RequestMapping("app-name")
      protected String getApplicationName() {
            //return applicationName;
            return env.getProperty("spring.application.name");
      }
```

Still if we run our application, we will get same output , which means we can get values defined in application.properties by autowiring Environment too.

Externalizing Configuration

Spring boot allows you to externalize your configuration properties. Above we saw, one way to do so using application.properties. There are various way to tell your application about properties required by your application i.e. it is not only limited to defining in application.properties .

Below are the ways you can define your properties. It is in order of preference which means , if same property is defined via two different ways , the one with higher precedence will be taken and the one with lower precedence value will be ignored.

1. Devtools global settings properties on your home directory (~/.spring-boot-devtools.properties when devtools is active).
2. @TestPropertySource annotations on your tests.
3. @SpringBootTest#properties annotation attribute on your tests.
4. **Command line arguments.**
5. Properties from SPRING_APPLICATION_JSON (inline JSON embedded in an environment variable or system property)
6. ServletConfig init parameters.
7. ServletContext init parameters.
8. JNDI attributes from java:comp/env.
9. Java System properties (System.getProperties()).
10. OS environment variables.
11. A RandomValuePropertySource that only has properties in random.*.
12. Profile-specific application properties outside of your packaged jar (application-{profile}.properties and YAML variants)
13. Profile-specific application properties packaged inside your jar (application-{profile}.properties and YAML variants)
14. Application properties outside of your packaged jar (application.properties and YAML variants).
15. **Application properties packaged inside your jar (application.properties and YAML variants).**
16. @PropertySource annotations on your @Configuration classes.
17. Default properties (specified using SpringApplication.setDefaultProperties).

The above list is taken from spring documentation present at https://docs.spring.io/spring-boot/docs/current/reference/html/boot-features-external-config.html

Tough it is good to know that there are several ways to define the properties which will be used for our application , it is hardly a chance that you will use all of them in our application . We will normally use them on need basis.

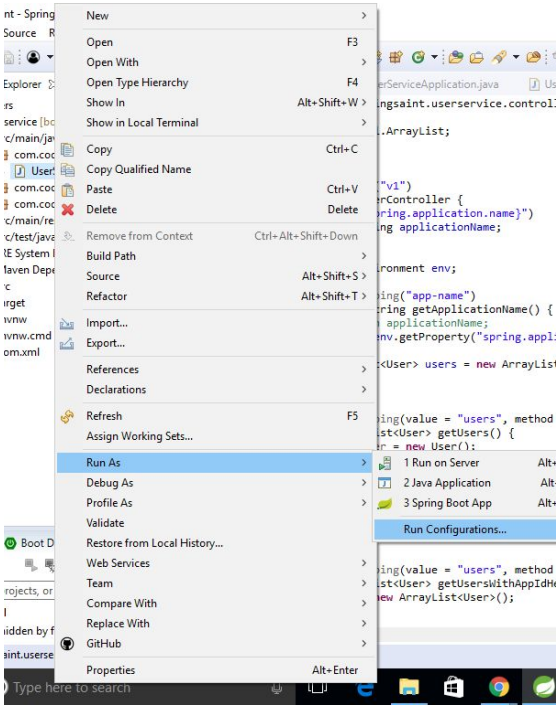
In order to verify the order of precedence , we can test it. In above list the number 4th (command line) and 15th (application.properties inside jar) are marked as bold. As per precedence order command line takes higher precedence over application.properties inside our jar file.

We already have application.properties with server.port =8081 , let's add a command line argument of same before starting application and verify if our server starts at different port.

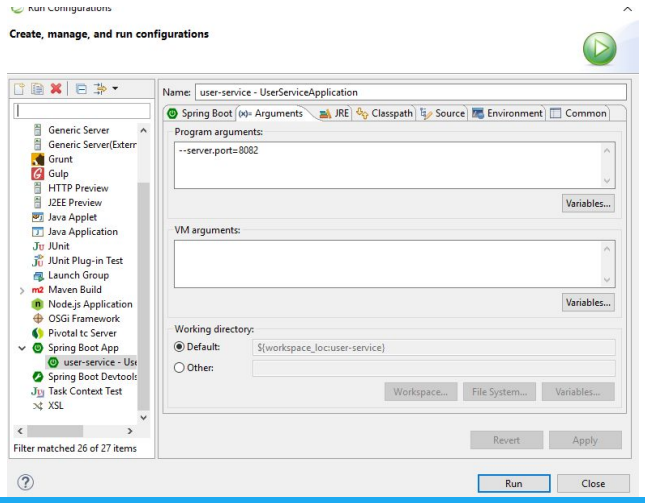
Currently our server starts at 8081 , say we want application to pick 8082 and ignore the value provided by application.properties (or application.yml) . We can provide command line arugument directly by invoking jar file made by build with command parameter like

java -jar user-service-0.0.1-SNAPSHOT.jar --server.port=8082

Since we are developing via an IDE , we can use IDE to add it for us , in STS or Eclipse you can right click on project , go to Run As -> Run Configuration



Inside Run Configuration , open Arguments tab and add --server.port =8082 as shown below.



You can now see our server now starting at 8082.

```
INFO 5688 --- [  restartedMain] c.c.userservice.UserServiceApplication   : Starting UserServiceApplication on DESKTOP-RL1NCCE wit
INFO 5688 --- [  restartedMain] c.c.userservice.UserServiceApplication   : No active profile set, falling back to default profile
INFO 5688 --- [  restartedMain] ationConfigEmbeddedWebApplicationContext : Refreshing org.springframework.boot.context.embedded.A
INFO 5688 --- [  restartedMain] s.b.c.e.t.TomcatEmbeddedServletContainer : Tomcat initialized with port(s): 8082 (http)
INFO 5688 --- [  restartedMain] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
INFO 5688 --- [  restartedMain] org.apache.catalina.core.StandardEngine  : Starting Servlet Engine: Apache Tomcat/8.5.23
INFO 5688 --- [ost-startStop-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
```

Now since we have already externalized configuration , we would see some other popular usage of configuration.

Configuring Random values

Many times we need to get random values ,for example you need to generate random pin for verification or a some random value for email verification. Spring provides easy way to do it.

RandomValuePropertySource is used to inject random values. RandomValuePropertySource can produce integers, longs, uuids or strings.

In application.properties we will add following :

```
app.random=${random.value}
my.random.integer=${random.int}
my.random.long=${random.long}
my.random.uuid=${random.uuid}
my.random.less.than.hundred=${random.int(100)}
my.random.within.range=${random.int[1000,9999]}
```

The above values are self explanatory . They are creating random values. In order to see if they work as expected let’s create a RequestMapping so that we access these value and display on UI /POSTMAN.

We will inject above defined properties via @Value annotation. Doing so we can also see that spring will keep the type safety in consideration. If we take a Integer value to inject an integer it will inject integer , Long for long and so on . Let’s create another controller CommonController as below with a RequestMapping.

```
@RestController
@RequestMapping("v1")
public class CommonController {
      @Value("${app.random}")
      private String random;
      @Value("${app.random.integer}")
      private Integer randomInteger;
      @Value("${app.random.long}")
      private Long randomLong;
      @Value("${app.random.uuid}")
      private String randomUUID;
      @Value("${app.random.less.than.hundred}")
      private Integer randomLessThanHundred;
      @Value("${app.random.within.range}")
      private Integer randomWithinRange;

      @RequestMapping("random-values")
      protected Map<String,Object>getRandomValues(){
                  Map<String,Object> randomValues= new
LinkedHashMap<String,Object>();
                  randomValues.put("random",random);
                  randomValues.put("randomInteger",randomInteger);
                  randomValues.put("randomLong",randomLong);
                  randomValues.put("randomUUID",randomUUID);

randomValues.put("randomLessThanHundred",randomLessThanHundred);
                  randomValues.put("randomWithinRange",randomWithinRange);

                  return randomValues;

      }
}
```

Here we can observe that we are using a Map to collect all values in our method and send them back. Once we hit http://localhost:8082/v1/random-values (Considering you are still giving port as command line parameter i.e.--server.port=8082) we will get following response.

```
{
    "random": "2b26227a3a78bbbac96617f094b613e6",
    "randomInteger": 728349517,
    "randomLong": -529828434181910766,
    "randomUUID": "c5b8aa23-5992-48c3-9aed-f7b9e0e223c7",
    "randomLessThanHundred": 5,
    "randomWithinRange": 7208
}
```

With above value , we can be assured that our RandomValuePropertySource is correctly working.

Property via command line

In above example we have seen how to send command line parameter as --server.port , We can pass any parameter via command line. The parameters which are sent via command line can be accessed using @Value annotation ,using Environment variable provided by Spring as discussed above. Interestingly you might want to disable your application taking command line arguments. There could be compelling reasons to do so, one of them could be security . If so, go ahead adding following line and no more command line parameters will be considered.

```
SpringApplication.setAddCommandLineProperties(false);
```

Profile Specific Properties

Spring provides profiles for environment based properties. For example Database related properties will change for Development , QA and Production environment. For such cases we can create property file like

application-{profile}.properties

Like application-dev.properties , application-qa.properties or application-prod.properties

By default , if not specified profile is “default”, You can see this happening in logs as well. Below is the screenshot of same :

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v1.5.9.RELEASE)

2018-03-01 14:21:50.579  INFO 232 --- [  restartedMain] c.c.userservice.UserServiceApplication   : Starting UserServiceApplication on QSDBASE7040C with PID 232 (C:\
2018-03-01 14:21:50.581  INFO 232 --- [  restartedMain] c.c.userservice.UserServiceApplication   : No active profile set, falling back to default profiles: default
2018-03-01 14:21:50.734  INFO 232 --- [  restartedMain] ationConfigEmbeddedWebApplicationContext : Refreshing org.springframework.boot.context.embedded.AnnotationCo
```

Consider a property like app.name defiled in application.properties and as well as application-{profile}.properties like application-dev.properties. If the profile is set to dev , the value for app.name in.properties will be considered. It will override the default one.

We will see profile in details in next chapter.

Placeholder in properties

Placeholders could be of great use when defining any properties file specially a long one with references from other properties. For example , see below properties

```
app.name=User Application

app.version= 1.0.0

app.description = ${app.name} - ${app.version} is an application to store user details.
```

If we add above properties in application.properties file , and try to fetch app.description property, It will refer app.name and app.version to create value for app.description. Let’s test it

We will add a Simple request mapping in our common controller and return these values via a map.

```
@Autowired
private Environment env;
@RequestMapping("app-info")
protected Map<String,String>appInfo(){
        Map<String,String>appInfo= new LinkedHashMap<String,String>();
        appInfo.put("name", env.getProperty("app.name"));
        appInfo.put("version", env.getProperty("app.version"));
        appInfo.put("description", env.getProperty("app.description"));
        return appInfo;
}
```

Now if we hit the URL http://localhost:8081/v1/app-info , we will get following response validation that placeholders are working. (*It can be noted that we have removed command line parameter of --server.port, that's why it is running on 8081 picking from properties file*)

```
{
    "name": "User Application",
    "version": "1.0.0",
    "description": "User Application - 1.0.0 is an application to store user details."
}
```

Here name and versions are replaced in description.

Using YAML file

Yaml as they are called Yet Another Markup Language is an alternative to properties file. It has extention of **".yml".** We can create application.yml file instead of application.properties.

Following it table for equivalent properties and YAML

application.properties

```
app.name=User Application
App.version: 1.0.0
app.description = ${app.name} - ${app.version} is an application to store user details.
app.developers.name[0]=Ray McFallen
app.developers.name[1]=John Smith
```

application.yml

```
app:
  name : User Application
  Version: 1.0.0
  description: ${app.name} - ${app.version} is an application to store user details.
  developers:
    name:
      - Ray McFallen
      - John Smith
```

As we can see yaml file is less repetitive than properties file in term of key namings. The another difference to be noticed is in list of elements (Array in above case) .

Yaml file is lot cleaner approach. With properties file we are creating profile specific properties file i.e. application-{profile}.properties . Which means for each profile there will be separate properties file. With yml , we can use three “-” like --- and write profile specific values in same file.

```
app:
  name : User Application
  Version: 1.0.0
---
spring:
  profiles: development
  Version: 1.0.40
---
spring:
  profiles: production
  Version: 1.0.3
```

Here version will change if application is started with profiles as development and production but by default (if no profile is given) it will pick 1.0.0 .

The biggest shortcoming of YAML is that you can’t use it with @PropertySource annotation. This annotation is used to load property files other named other than application*properties . PropertySource annotation is used for externalizing properties.

Type Safe Configuration Properties

At times too many of @Value annotation is annoying. Think of a file where we require hundreds of property to be read from yaml or properties file. The number of variable we will be declaring would be way too many.

This problem looks more ugly in case if we require similar kind (read group) of properties. For example in case of properties related to Database or Security related configurations. Below is such a example :

```
# application.properties
#For using @ConfigurationProperties
application.name=User Application
application.version=1.0.0
application.description = ${app.name} - ${app.version} is an application to store user details
.application.security.enabled=true
application.security.user=admin
application.security.email= unk@unk.com
application.security.roles[0]=USER
application.security.roles[1]=ADMIN
```

Here if we try to read variables , tough all variables are related to app(which is at top level of hierarchy), we will end up creating many variable to read each property from YAML , this might even make our class look ugly if number of properties increases significantly.

Spring provides a simple way to help us. We can create a class and bind all the properties from configuration files like YAML or properties . We can create a class with @ConfigurationProperties

@ConfigurationProperties

For above configuration let's create a class and verify that it works.

```
@ConfigurationProperties(prefix = "application")
@Component
public class AppConfig {
        private String name;
        private String version;
        private String description;
        private Security security;
//getters and setters
}
```

We can create a security class and use it inside AppSecurity , values corresponding to app.security will be set inside it.

Security.java

```
public class Security {
      private String user;
      private String email;
      private String [] roles;
//getters and setters
}
```

Now let's autowire AppConfig class in our CommonController and create a request mapping to see that it works.

Inside CommonController.java

Add a Autowired variable for AppConfig

```
@Autowired
private AppConfig appConfig;
```

And then add a request mapping which returns AppConfig. If bindings are correct ,it will return all the values.

```
@RequestMapping("app-config")
protected AppConfig appConfig(){
       return appConfig;
}
```

Once done , lets verify if the entire code is working well . Just hit URL :

http://localhost:8081/v1/app-config

You should be getting following response.


```
{
  "name": "User Application",
  "version": "1.0.0",
  "description": "User Application - 1.0.0 is an application to store user details",
  "security": {
    "user": "admin",
    "email": "unk@unk.com",
    "roles": [
      "USER",
      "ADMIN"
    ]
  }
}
```

This shows how easily we can have a type safe and less verbose property file reading using , @ConfigurationProperties.

4. Profiles

While developing any application on any framework, one of the most common problems you would have came across is managing configurations for it . The configuration for local setup will be different from the configuration at DEV/INT environment, which is again different from QA/TESTING environment and of course it will different from PRODUCTION. There is always things working at local of a developer but failing at testers environment. Many of the times the reason for such problems are difference in configurations.

To understand it a little better and in one of the most common scenario , development database is different from QA or production database. While we have learned to externalize our properties using properties or YAML configuration file, We don't expect to modify it everytime we ship our product for different environments.

Here comes profiles for our help, think it as a tool to help you from segregating the configurations based on environments or parameters of your choice. It makes the job easier since everytime you just have to specify the environment and spring will select the properties based on your choice of profiles.

Let’s see it in action.

Using Properties file for profiles.

Let’s have situation where we have different email address to be used for notification , at development and production has different group mailing address. If we hardcode it in Java file , obviously it looks bad. Even single property file where we can change the values are not so helpful as every time before deployment or creating JAR for specific deployment (DEV or PROD) we will have to change it.

What if we create multiple property file for different environment and Spring picks correct one just before application starts. This looks great and Spring provides us to do so,

Let’s think we have two application properties file with name application-dev.properties and application-qa.properties. Below are the content for them:

application-dev.properties

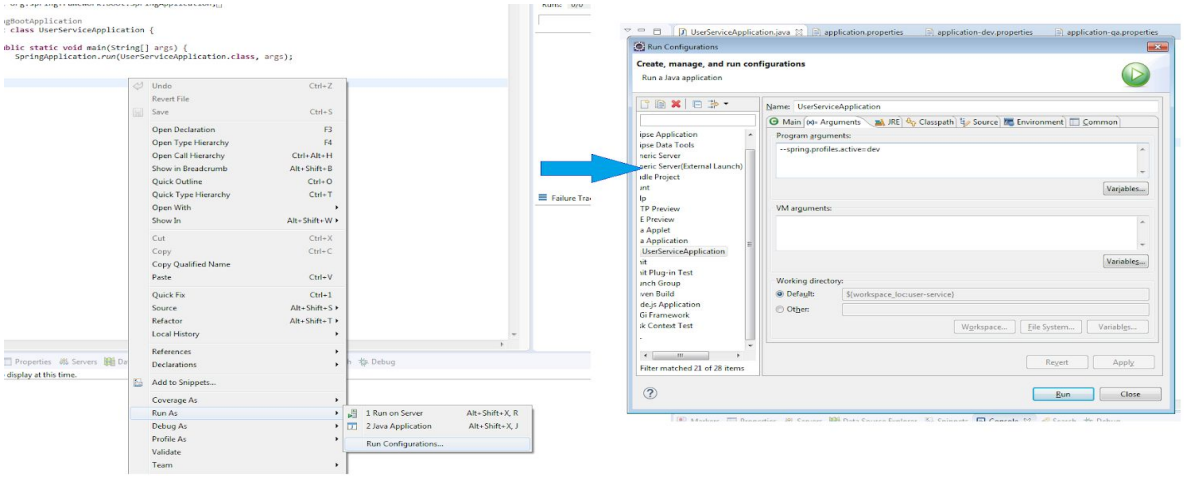
```
application.security.user=admin-dev
application.security.email= unk-dev@unk.com
application.security.roles[0]=USER
application.security.roles[1]=ADMIN
```

application-qa.properties

```
application.security.user=admin-qa
application.security.email= unk-qa@unk.com
application.security.roles[0]=USER
application.security.roles[1]=ADMIN
```

Notice that still we have these properties present in appilcation.yml along with all other properties used till now. It contains all non profile specific values as well as those written as profiles based ones which are listed above in application-dev.properties and application-qa.properties.

Now we will tell Spring boot to enable profiles. In order to do it we can either define property *spring.profiles.active* in application.properties or use other ways of configuring properties as discussed in previous section. Defining in application.properties is a brittle approach asking to change property file every time you start the application.One other popular way is to pass as program or VM argument while starting application. We will pass it as a program argument. If you are running from IDE find the ways of passing program argument while start. In eclipse/STS , you can right click on Main class (UserServiceApplication.java) and select “Run As” under which Run Configuration can be found. Add --spring.profiles.active=dev as Program Argument as shown in figure.

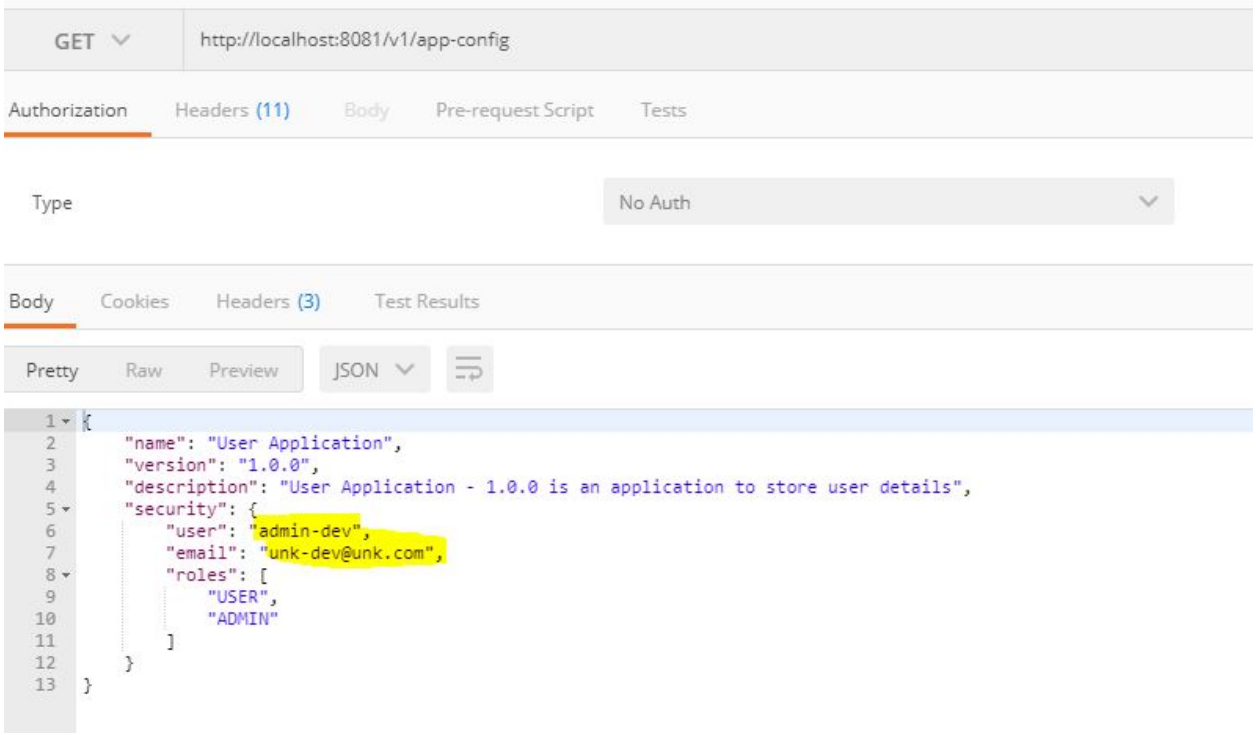


Run your application and in console you can see spring selecting profile as dev.

```
JserServiceApplication [Java Application] C:\Program Files\Java\jdk1.8.0_25\bin\javaw.exe (Jul 6, 2018, 12:14:54 PM)

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.0.1.RELEASE)

2018-07-06 12:14:57.799  INFO 7924 --- [  restartedMain] c.c.t.s.u.UserServiceApplication         : Starting UserServiceApplication on QSDBASE7040C
2018-07-06 12:14:57.806  INFO 7924 --- [  restartedMain] c.c.t.s.u.UserServiceApplication         : The following profiles are active: dev
2018-07-06 12:14:58.011  INFO 7924 --- [  restartedMain] ConfigServletWebServerApplicationContext : Refreshing org.springframework.boot.web.servlet
2018-07-06 12:15:01.326  INFO 7924 --- [  restartedMain] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.transaction.annotatio
2018-07-06 12:15:02.259  INFO 7924 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8081 (http)
```

Now we will hit same URL for app config which we were using in our last exercise. We can notice there is a change security user and email parameters and the one from application-dev has been picked. The values of application.properties are overridden.

URL : http://localhost:8081/v1/app-config



a. Adding Active Profiles

Say we want to have separate property files for database configuration. Currently we have all the database related properties in application.properties . In order to have separation on concern or for any reason , if we require multiple profiles to be used for our application we can pass multiple profiles as comma separated parameter viz. --spring.profiles.active=dev,db , else we can use spring.profiles.include in any of our application.properties file which we are using . Let's try it.

We will :-

comment the properties writted for database in application.properties file.

b. Add a new file application-db.properties .
c. Add our properties there , changing database name to verify if changes are been reflected.

d. Add following property In application.properties file

   i. spring.profiles.include=db

application.properties

```
spring.profiles.include=db
# H2
spring.h2.console.enabled=true
spring.h2.console.path=/h2
# Datasource
#spring.datasource.url=jdbc:h2:file:~/user
#spring.datasource.username=sa
#spring.datasource.password=
#spring.datasource.driver-class-name=org.h2.Driver
```

Notice we have commented all datasource related properties and added spring.profiles.include=db

Now look at content of application-db.properties

```
spring.datasource.url=jdbc:h2:file:~/appuser
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
```

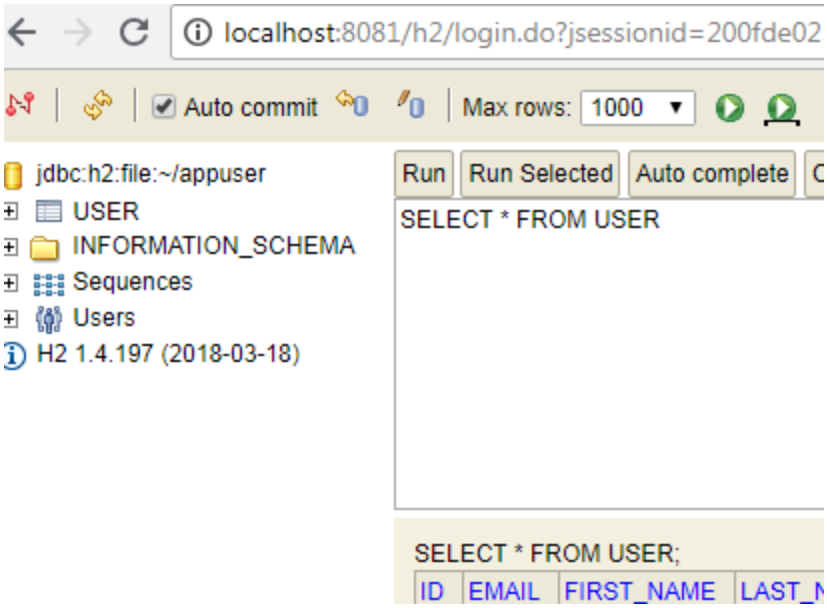
Now if we restart our app we can see console picking multiple profile and our database is configured at `spring.datasource.url=jdbc:h2:file:~/appuser` , `If you see we have modified this property and are using appuser at end rather user.`

`Console:`

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.0.1.RELEASE)

2018-07-06 14:59:35.434  INFO 7924 --- [  restartedMain] c.c.t.s.u.UserServiceApplication         : Starting UserServiceApplication on QSDBASE7040C with PID 7924 (C:\Users\c00349
2018-07-06 14:59:35.434  INFO 7924 --- [  restartedMain] c.c.t.s.u.UserServiceApplication         : The following profiles are active: db,dev
2018-07-06 14:59:35.438  INFO 7924 --- [  restartedMain] ConfigServletWebServerApplicationContext : Refreshing org.springframework.boot.web.servlet.context.AnnotationConfigServle
2018-07-06 14:59:36.077  INFO 7924 --- [  restartedMain] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.transaction.annotation.ProxyTransactionManagementCon
2018-07-06 14:59:36.297  INFO 7924 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8081 (http)
```

We can see it has picked up multiple profiles dev and db. Now if we hit http://localhost:8081/h2/

We will see it having the tables being created and a indication of working with multiple profiles



5. Logging

Logging is one of the most common and important features over different frameworks. There are many famous logging frameworks like Log4J , Sl4J ,logback etc. Spring Boot uses Common Loggin for all internal loggint it does. At the same time it's on developer to decide which logging framework suits his need, THe default supported ones are are **Java Util Logging, Log4J2, and Logback.**

If we use startes Spring by default uses Logback.

Format of Logging

Look at the below line lines generated by our application as log

```
4:59:35.434  INFO 7924 --- [  restartedMain] c.c.t.s.u.UserServiceApplication         : The following
profiles are active: db,dev
Tomcat initialized with port(s): 8081 (http)
```

It contains :-

he following items are output:

1. Date and Time: Millisecond precision.
2. Log Level like ERROR, WARN, INFO, DEBUG, or TRACE.
3. Process ID.
4. A --- separator to differentiate between actual message and metadata which we discussed above
5. Thread name: Enclosed in square brackets (may be truncated for console output).
6. Logger name: Abbreviated class name
7. The log message.

Console Output

By default all the spring default implementation are enabled to write on console with an option to write in a file. As default, all messages set at level ERROR, WARN and INFO are logged. It will not be logging debug by itself until we explicitly inform it to do so. If we want to log at debug level we can simply add debug as parameter while running application as below

Java -jar app.jar --debug

We can also specify parameter as `debug=true` in application.properties.

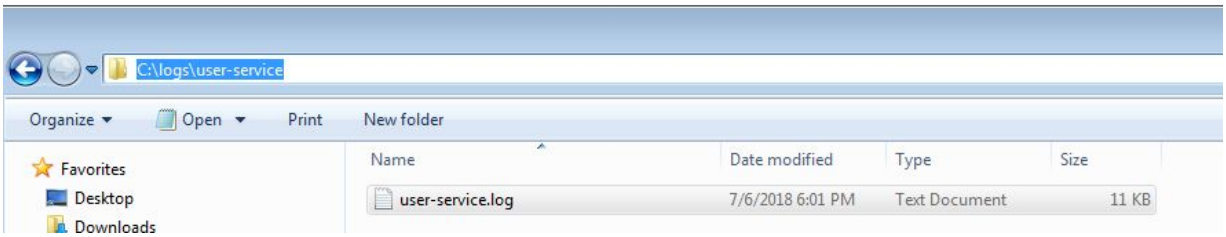
a. File Output

If we need (and obviously we will need) to log in a file we can use logging.file or logging.path properties.

Let's see what happens if we add these properties in application.properties file

```
logging.path=c:/logs/user-service
logging.file=${logging.path}/user-service.log
```

Now logging.path will be resolved and we can see a file named user-service.log at same location.



b. Log Level

We can log different packages at different level , In order to do so we just have to add property for each such packages . Here logging.level is followed by package ,look at below example.

```
logging.level.root=WARN
logging.level.org.springframework.web=DEBUG
```

Here root (default) logging level is warn but anything logged from org.springframework.web is at level DEBUG.

c. Profile specific configurations

As we have already seen that we can define parameters for creating log files , we can also have a log configuration file at any location. For example, think of a situation where we would like a log4j or logback configuration xml at certain disk location. The system should be reading this configuration file for applications log related parameters.

We can define logging.path in application.properties or pass it as a parameter viz. --logging.config=/var/log/logback-spring.xml

At first we will create a logback-spring.xml file and keep it at c:/logs/config/ folder. Below are it's content.

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>
       %d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
      </pattern>
    </encoder>
  </appender>
  <root level="info">
    <appender-ref ref="STDOUT"/>
  </root>
</configuration>
```

Now, we are can add a property in application.property telling the path of logback-spring.xml

logging.config=c:/logs/config/logback-spring.xml

With this if we restart our app the configuration and style of our log files will be as per the logback-spring.xml added at locatlon *c:/logs/config/logback-spring.xml* .

Sometimes we want that our log files should be configured differently for different profiles. Just think if we want to enable DEBUG at DEV or STAGING instance but it would not be good to log at DEBUG level at production and overwhelming the loggers at PROD.

We can have different files for different situations but that could be headache later if you want to manage. How great it would be if one file can serve all of the purpose , the file should be able to tell system that for certain profile these are valid log configurations. Spring enables this using profile specific configuration.

We can modify logback-spring.xml and make it such that for each profile we have a different set of configuration. Let's see below example

```
<springProfile name="staging">
      <!-- configuration to be enabled when the "staging" profile is active
```

```
-->
</springProfile>

<springProfile name="dev, staging">
      <!-- configuration to be enabled when the "dev" or "staging" profiles
are active -->
</springProfile>

<springProfile name="!production">
      <!-- configuration to be enabled when the "production" profile is not
active -->
</springProfile>
```

If we use such a configuration at logback-spring.xml , we can see that our profiles are making impact to logging configuration.

d. Environment properties

The files like `logback-spring.xml` in our above example can be really more fun if we make it more configurable , What if we can add some dynamic parameters.

Spring allows you to use Environment property to be used with <springProperty> tag. With this you can access values from spring's application.properties to logback configuration.

Try using this configuration and it will still work.

In spring application.properties , we have a property spring.application.name=user-service , Let's try to use it with our logback-spring.xml, We can also have a fallback using defaultValue ,

From above logback config we will modify as following

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
<springProperty scope="context" name="appName"
source="spring.application.name"
            defaultValue="USER SERVICE"/>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>
       %d{yyyy-MM-dd HH:mm:ss}[ ${appName} ] [%thread] %-5level %logger{36}
- %msg%n
      </pattern>
    </encoder>
  </appender>
  <root level="info">
    <appender-ref ref="STDOUT"/>
  </root>
</configuration>
```

With above configuration we can see that our log will pick spring.application.name property from application.properties and now our logs will look like below, Notice user-service added to logs

```
2018-07-30 15:43:00[ user-service ] [restartedMain] INFO
c.c.t.s.u.UserServiceApplication - The following profiles are active: db
```

6. Developing a web application

a. Introduction to Web MVC Framework

World wide web has been around for years. Spring supports web application development. Most of the project with spring boot will have spring-boot-starter-web as dependency by default. It also contains embedded server as Tomcat, Undertow and Jetty. Spring Boot will help us to get started quickly on any project.

Spring Web MVC or Spring MVC will require common annotation. Many of it we have already gone through viz. `@Controllers`, `@RestControllers`, `@RequestMapping` etc. , We also went through custom Spring Boot annotations like @SpringBootApplication.

Now we will learn some more interesting annotations and features which makes our application more robust

`EnableAutoConfiguration`.

b. Spring MVC Autoconfiguration

Spring Boot provides Auto configurations facilities for Spring MVC. It is due do auto configurations that we find defaults being enabled in Spring Boot applications. Spring Boot just looks the classpath and automatically adds defaults. For example , if it finds HSQLDB on classpath it will automatically add dataSource and configures in-memory database.

Spring Boot never stops you from deciding configurations. Despite of the fat that it applied default configurations , it never comes in your way to override them. The time you add your set of configurations defaults configuration backs away.

Disabling auto configuration

If you wish to disable autoconfiguration you can simple exclude them. In order to exclude any autoconfiguration , you can use @EnableAutoConfiguration tag with exclude parameter. In following code sample we will see how can we remove DataSource related configurations by excluding DataSourceAutoConfiguration class.

```
@Configuration
@EnableAutoConfiguration(exclude={DataSourceAutoConfiguration.class})
public class MyCustomConfiguration {
}
```

Apart from writing a custom configuration we can also achieve it by using `spring.autoconfigure.exclude` tag in property or yml file.

c. HttpMessageConverter

Spring boot contains an interface HttpMessageConverter to convert HTTP messages and HTTP responses. HttpMessageConverters helps to convert Object to JSON or XML and vice versa using Jackson Library.

By default following converters are supported if classpath includes required library.

`ByteArrayHttpMessageConverter` : Converts ByteArray

`StringHttpMessageConverter` : Converts String

`ResourceHttpMessageConverter` : org.springframework.core.io.Resource for any type of octet stream

`SourceHttpMessageConverter`: converts javax.xml.transform.Source

`FormHttpMessageConverter`: read and write the "application/x-www-form-urlencoded" media type as MultiValueMap<String, String>

`Jaxb2RootElementHttpMessageConverter` – To convert Java objects to/from XML if JAX2B is on classpath

`MappingJackson2HttpMessageConverter` – To convert JSON if Jackson2 is on classpath

`MappingJacksonHttpMessageConverter` – To convert JSON if Jackson is on classpath

`AtomFeedHttpMessageConverter` – To convert Atom feeds if Rome is on classpath

`RssChannelHttpMessageConverter` – To convert RSS feeds if Rome is on classpath